Debugging Programs with PDB

Part 1. Start debug session using "**python -m pdb sort.py**"

- **b(reak)**: "**b 5**" - set break on line 5. "**b func**" - set break at 1st executable line of func. "**b**" no args - list all breakpoints. "**cl 3**" - clear breakpoint w/id 3 (NOT on line 3).
- **n(ext)** - execute current line, stop at next one. Func calls run to completion w/out stopping (no stepping into funcs). **BARE ENTER REPEATS LAST COMMAND**, e.g. "n".
- **s(tep)** - *step into func*: execute current line, stop at 1st line of any func it calls. Ex. print(merge_sort(arr)) - merge_sort is the 1st func called, print is 2nd. If "s" gets you to line w/decorator, do "s" again to get into func itself.
- **s(tep)** - *at the next recursion point* - step into the next recursion, down the recursion stack. **u(p)** and **d(own)** help move up and down the recursion stack (to print values).
- **c(ontinue)** - run freely until next breakpoint, or program end.
- **r(eturn)** - continue execution until the current function returns. Need to follow up with "n" to be able to get the returned value. Helpful if there are several returns in one func.
- **l(ist)** - displays 11 lines around current line. "**l 5**" - display 11 lines starting from line 5. "l 5, 9" - display lines 5-9
- **var1** - prints value of var1 if var1 is not a pdb command (b, s, c, etc.). If it is, do "**p var1**".
- **p(rint)** - prints 1 or more vars. Syntax: p var1, var2, ... (use commas). "pp" for pretty print
- **!Python command** - e.g. "**!c**" prints value of var c. Equiv. of "**p c**" since c is a pdb command.
- **display <Python expr.>** - prints value of expr. (e.g. var1) **repeatedly** for each execution stop (n/c/s/r/breakpoint). Stops after undisplay. "p" prints only once. Use copy arr[:] here (not arr)
- Set **commands** to be executed **when reaching breakpoint 4**. Notice change to (**com**), and must close the command block with "**end**":
  (**Pdb**) commands 4 *# enter commands block*
  (**com**) print(f"The total value as of now is {total}") *# Python code or pdb commands*
  (**com**) end *# exit commands block*
- **Print ALL vars**: pp {k: v for k, v in locals().items() if not k.startswith('_')} (also see snoop below)
- **w** - where: print the [recursion] stack trace, > marks the current frame. **q** - quit.
- **a** - print current function's arguments.

Part 2. Start debug session using "**python sort.py**" + "**breakpoint()**" in code

- Add "**breakpoint()**" OR "**import pdb; pdb.set_trace()**" (before Python 3.7) and the pdb debug session will open at the next line in code with access to all vars for debugging.
- Beneficial for conditional stopping (if expr: breakpoint(), e.g. when counter==500) or inside except in try/except.

pdb++: pip install pdbpp. All pdb commands work. Main additional benefit: **sticky mode** — persistent source view that updates as you step, like a mini-TUI (+ other enhancements).

Part 3. Using Python Library **snoop**

Auto-logs / prints every line executed in func + precise var modifications w/out print statements!

pip install snoop

```
import snoop

@snoop          # to watch vars in addition to local ones: @snoop(watch=('arr[mid]',))
def calculate_sum(numbers):
    func body
```